ANTIGRAVITY TELEGRAM AGENT v5.1

Complete Operational Manual: How to Use, Communicate, Customize & Master the System

Target Engine: Google Antigravity CLI (agy v1.4.12+) | **Architecture:** Warm Stream-JSON Engine | **Status:** Production Ready

> ***Executive Overview:*** *Antigravity Telegram Agent v5.1 bridges the gap between state-of-the-art autonomous AI engineering engines and the Telegram messaging interface. It enables software engineers and technical leaders to supervise codebases, execute terminal workflows, manage servers, schedule autonomous background routines, and orchestrate specialized subagent swarms from any mobile device or workstation with sub-second response times.*

1. System Architecture & The Zero Cold-Start Engine

Standard CLI wrappers spawn a brand-new process on every single incoming prompt. In Google Antigravity CLI, initializing the runtime, loading configurations, authenticating Google identity, and registering tool catalogs takes **12 to 18 seconds** per query. Furthermore, a fresh process flushes Google AI's prompt cache, significantly increasing token costs and latency.

Antigravity Telegram Agent v5.1 solves this via its Warm Stream-JSON Architecture:

- **Warm Persistent Process (agy_session.py):** A single Antigravity CLI process runs indefinitely in JSON streaming mode. User turns are delivered via standard input (stdin) and events are consumed from standard output (stdout) with zero initialization delay.
- **Prompt Cache Preservation:** System instructions, repository trees, and dynamic MCP schemas remain continuously cached in Google AI's memory, achieving up to 90% cost savings on recurring context.
- **Thread-Safe Asynchronous Turn Queue:** Inbound user messages, scheduled automation tasks, and subagent callbacks are strictly serialized to prevent race conditions or pipe deadlocks.
- **Live Multi-Stage Event Parsing:** The gateway decodes stream events in real time, separating internal chain-of-thought, terminal shell commands, file edits, and multi-agent delegation.

2. Operating the System via Telegram

Interacting with the system feels as natural as chatting with a senior staff engineer on Telegram. The interface is fortified with dynamic interactive controls and visual feedback.

Interactive Thinking Bubble & Live Streaming

While the AI works, Telegram displays an animated status bubble with an active spinner (■, ■, ■). You witness the agent's thought trajectory and tool invocations as they occur. An inline **[■ Abort]** button lets you cancel long-running loops or misdirected commands instantly.

Essential Telegram Command Matrix

Command	Scope & Arguments	Operational Function
`/start`	None	Initializes session, claims admin identity, opens dashboard.
`/settings`	None	Interactive control panel: Model, Effort, Mode, Sandboxing.
`/usage`	None	Real-time visual token gauges, cooldown timers, API ping test.
`/models`	[model_name]	Switch between Gemini 3.8 Flash, Gemini 2.5 Pro, Argon 4, Claude 3.7.
`/effort`	[auto|low|med|high|max]	Adjusts thinking reasoning budget. High/Max for architecture.
`/mode`	[auto-approve|accept-edits|plan]	Execution autonomy: Full auto, manual approvals, or plan-only.
`/tasks`	None	Inspect and manage scheduled automations, timers, and cron jobs.
`/agents`	None	Inspect, create, activate, export (ZIP), and import subagent swarms.
`/mcp`	None	Inspect connected Model Context Protocol servers and diagnostic status.
`/clear`	None	Flushes current conversational context while preserving long-term memory.
`/abort`	None	Immediately terminates active CLI execution and resets turn state.

Multimedia & Bidirectional File Handling

The agent seamlessly ingests and generates diverse data formats:

- **Inbound Media:** Drop screenshots of error traces or Figma mockups; the AI performs visual multimodal analysis. Send voice notes while on the move; the bot transcribes speech into text instructions automatically. Upload code files or ZIP archives to mount them directly into the workspace.
- **Outbound Media:** When the AI compiles a PDF report, renders an architecture diagram, creates a build artifact, or exports an archive, it automatically transmits the deliverable directly to your Telegram chat using tg_send.py --file <path>.

3. Conversational Engineering: How to Talk to the AI

Unlike generic consumer chatbots, Antigravity Telegram Agent is an autonomous engineering agent connected to your local filesystem, execution terminal, and git repositories. Communicating with precision yields extraordinary results.

The Three Core Rules of Agent Prompting

1. **Define Outcomes, Scope, and Boundaries:** State what files should be modified and what should remain untouched. Specify the test framework or coding style required.
2. **Demand Grounded Evidence & Verification:** Instruct the agent to run unit tests, check linter output, or verify HTTP responses before declaring completion. (e.g. *'Implement the endpoint and run pytest to confirm 100% pass rate.'*)
3. **Leverage Incremental Modularity:** For extensive architectural shifts, break tasks down into specification, implementation, and verification phases.

Specialized Operational Workflow Modes

The system ships with pre-engineered workflow commands that configure the reasoning engine on the fly:

Mode / Slash Command	When to Use & Behavior
`/plan [goal]`	**Architectural Planning:** Analyzes the codebase and drafts a step-by-step verifiable blueprint without touching source code.
`/grill_me [concept]`	**Interactive Interview:** Conducts a disciplined Socratic interview. Asks targeted architectural questions one by one before any code is written.
`/goal [task]`	**Exhaustive Autonomous Execution:** Operates in deep pursuit mode. Verifies every step, writes unit tests, and does not stop until fully achieved.
`/boost [problem]`	**Maximum Reasoning & Auditing:** Sets thinking effort to maximum, inspects edge cases, and cross-verifies logic using multi-perspective reasoning.
`/teamwork [mission]`	**Multi-Agent Swarm Orchestration:** Deploys subagents in parallel to divide and conquer research, implementation, and QA simultaneously.

4. Customization & Personalization Mastery

Antigravity Telegram Agent v5.1 provides unprecedented depth in personalizing behavior, personality, interpersonal speech rules, and specialized team structures.

A. Person Profiles & Speech Tone Rules (mem.py person)

You can teach the agent exactly who individuals are, their relationship to you, and **how it must speak to them** (formal vs casual, French tutoiement vs vouvoiement):

```
# Register Primary Owner with direct, technical tutoiement:
python mem.py person add "Ange" --role "Lead Architect" --relation "Owner" \
--tone "Tutoiement direct, concis, amical et technique" --aliases "ange,admin"

# Register an Executive Client requiring strict formal vouvoiement:
python mem.py person add "M. Dupont" --role "Chief Technology Officer" --relation "Client" \
--tone "Vouvoiement formel, poli, rigoureux et respectueux"
```

B. Social Graph & Relational Mapping (mem.py relate)

Track relationships between team members, partners, and contractors so the AI understands organizational context:

```
python mem.py relate "Ange" "co-founder" "Marc" --details "Co-owners of project Alpha"
python mem.py relate "Marc" "manages" "Sophie" --details "Frontend engineering lead"
python mem.py relate list
```

C. Creating Specialized Subagents (.agents/agents/*.md)

Subagents are defined as clean Markdown manifests with YAML frontmatter. Create them manually or say: *'Create a specialist subagent for PostgreSQL database indexing and query tuning'*:

```
---
name: db_architect
description: Senior PostgreSQL database architect and query optimization specialist.
model: gemini-2.5-pro
tools: [read_file, write_file, edit_file, run_command]
skills: [database-architect, sql-optimization]
---
You are an elite database engineer. Analyze EXPLAIN ANALYZE traces, design B-Tree/GIN indexes,
prevent N+1 query patterns, and ensure strict ACID transactional integrity.
```

5. Deep Capabilities & Advanced Autonomous Features

Antigravity Telegram Agent v5.1 represents the cutting edge in autonomous developer tools. Below are the primary subsystems that provide its industry-leading capabilities:

1. Long-Term Hybrid Vector Memory (mem.py / memory.db)

Backed by SQLite and FastEmbed semantic embeddings, the agent retains persistent knowledge across chat resets and reboots. Categories include: profile (identity, hardware), preferences (formatting, frameworks), projet (git repos, active milestones), infra (IPs, Docker containers, ports), and rules (permanent operating constraints).

2. Native Background Task Scheduler & Autonomous Automations (tasks.py)

The agent includes a persistent daemon worker running an independent SQLite task queue. Schedule tasks effortlessly using plain English, relative intervals, or cron schedules:

```
# One-shot reminder in 45 minutes:
python tasks.py add "Check staging build and verify logs" --when "in 45m" --name "check_staging"

# Recurring health-check every 3 hours:
python tasks.py add "Verify Docker containers status and report anomalies" --when "every 3h"

# Daily executive morning briefing at 08:30 AM:
python tasks.py add "Generate daily agenda, server status, and AI news brief" --when "08:30"
```

3. Official Google Antigravity Quotas & Cross-Family Fallback Ladder

Google Antigravity enforces strict rolling quotas for premium reasoning models. The gateway parses exact reset cooldowns from API headers and manages an automatic fallback ladder:

> ***Quota Fallback Sequence:***
> *Claude 3.7 Sonnet ➔ Claude 3.5 Sonnet ➔ Gemini 2.5 Pro / Argon 4 ➔ Gemini 3.8 Flash*
> *Result: Your workflows, deployments, and debugging sessions never freeze or fail due to credit exhaustion.*

4. Group Chat Stealth Mode & Anti-Prompt Injection

When added to team or community Telegram groups, the bot automatically engages Stealth Observation Mode. Group members cannot trigger subagent installation, file writes, terminal execution, or prompt injections. The bot only executes privileged commands when explicitly invoked by the authenticated Administrator.

5. Zero-Data-Loss Self-Updater (/update)

The agent monitors official upstream updates on GitHub. When /update is triggered, it creates an atomic backup of .env, settings.json, memory.db, and .agents/, pulls the new release, executes migrations, and hot-reloads the daemon without dropping active Telegram connections.

6. Troubleshooting, Best Practices & Quick Reference

Symptom / Scenario	Diagnostic Cause	Prescribed Solution
`Bot ignores messages`	Sender ID does not match ALLOWED_USER_ID.	Check your Telegram user ID via @userinfobot and update .env.
`Claude quota exhausted`	Rolling token quota limit reached in Google Antigravity.	Agent automatically uses Gemini Pro. View countdown in /usage.
`Process looping or stuck`	Subprocess waiting on blocking terminal prompt.	Send /abort or tap the inline ■ Abort button.
`Reset context cleanly`	Context window full of obsolete debugging history.	Send /clear. Long-term memory and person facts remain preserved.

Security & Responsibility Notice: *Antigravity Telegram Agent v5.1 has full terminal and filesystem access according to your configured permissions. Keep your TELEGRAM_BOT_TOKEN strictly private, enforce ALLOWED_USER_ID, and keep workspace sandboxing enabled when running in shared environments.*